Universidad de Alicante
Escuela Politécnica Superior

Técnicas Avanzadas de Robótica (TAR)

Práctica 1

Bases de ROS 2 (Humble)

Autor:
Hernández Delgado, Jaime

Curso: 2025–2026

Repositorio GitHub:
https://github.com/nickhernd/TAR

3 de marzo de 2026

Índice general

1. Parte 1: Primeros Pasos **2**
- 1.1. Creación del espacio de trabajo . . . 2
- 1.2. Ejemplo de un entorno: Nodos y Topics . . . 2
- 1.3. Servicios . . . 3
- 1.4. Ejercicios . . . 4
  - 1.4.1. Ejercicio 1: Publicar mensaje al topic `/chatter` . . . 4
  - 1.4.2. Ejercicio 2: Modificar el servicio suma . . . 4

2. Parte 2: Nivel Principiante **5**
- 2.1. Crear un Paquete . . . 5
- 2.2. Crear un Mensaje . . . 5
- 2.3. Ejercicios . . . 6
  - 2.3.1. Ejercicio 1: Paquete `p2pkg` y mensaje `P2pkg_mensaje` . . . 6
  - 2.3.2. Ejercicio 2: Nodos publisher y subscriber en `p2pkg` . . . 7

3. Parte 3: Pasando a Nivel Intermedio **8**
- 3.1. Crear un archivo launch . . . 8
- 3.2. Acciones . . . 8
- 3.3. Ejercicios . . . 9
  - 3.3.1. Ejercicio 1: `launch_ejercicio3.py` . . . 9
  - 3.3.2. Ejercicio 2: `launch_ejercicio4.py` con namespace y remapping 9
  - 3.3.3. Ejercicio 3: Acción `ejFibonacci.action` . . . 9
  - 3.3.4. Ejercicio 4: Servidor `ejercicios_fibServer.py` . . . 9
  - 3.3.5. Ejercicio 5: Cliente `ejercicios_fibClient.py` . . . 9
  - 3.3.6. Ejercicio 6: Servicio `service_temp` (conversión de temperatura) 10
  - 3.3.7. Ejercicio 7: Acción `battery_act` (simulación de batería) . . . 10

1. Parte 1: Primeros Pasos

1.1 Creación del espacio de trabajo

En esta sección se describe el proceso de creación y configuración del espacio de trabajo ROS 2 (`ros2_ws`), así como la instalación del entorno Docker con ROS 2 Humble.

1.2 Ejemplo de un entorno: Nodos y Topics

Pregunta 1

¿Qué podemos saber de este nodo? (Suscriptores, topics)

Respuesta:

Pregunta 2

¿Qué podemos saber sobre el topic que publica el nodo que acabamos de lanzar?

Respuesta:

Pregunta 3

¿Qué podemos saber sobre el nuevo nodo `/listener`?

Respuesta:

Pregunta 4

¿Qué ha sucedido ahora en el topic `/chatter`?

Respuesta:

Pregunta 5

¿Qué podemos saber a partir del gráfico de `rqt_graph`*?*

Respuesta:

[Captura rqt_graph]

Figura 1.1: Gráfico `rqt_graph` con los nodos `/talker` y `/listener`.

1.3 Servicios

Pregunta 6

¿Qué puedes comentar acerca de este servicio?

Respuesta:

Pregunta 7

Al lanzar el servidor y ejecutar `ros2 service list` *¿Qué servicios aparecen? ¿Cómo se ha creado el servicio* `minimal_service` *y cuál es su cometido?*

Respuesta:

Pregunta 8

¿Cómo puedo saber cuáles son los parámetros de la "petición" y de la "respuesta" de un servicio? ¿De qué tipología son estos parámetros?

Respuesta:

```
ros2 interface show <tipo_servicio>
```

Listing 1.1: Mostrar interfaz del servicio con ros2 interface show.

1.4 Ejercicios

1.4.1 Ejercicio 1: Publicar mensaje al topic `/chatter`

Publica desde la terminal un mensaje al topic `/chatter` para que sea leído por el nodo `/listener`.

Comandos utilizados:

```
ros2 topic pub /chatter std_msgs/msg/String "data: 'Tu mensaje
    aquí'"
```

Listing 1.2: Publicación manual en el topic /chatter.

1.4.2 Ejercicio 2: Modificar el servicio suma

Cambia el servicio `servicio_suma` para que calcule: $a + b - c \cdot d \div e$*, siendo* a, b, c, d, e *valores enteros (`int64`) y el resultado en `float64`.*

Descripción de los cambios:

Archivos modificados:

- `interfaz_servicio_suma/srv/...` – Modificación de la interfaz del servicio.
- `servicio_suma/...` – Modificación del servidor y cliente.

2. Parte 2: Nivel Principiante

2.1 Crear un Paquete

Pregunta 9

¿Qué comando me indica todas las dependencias que puedo usar en mis paquetes?

Respuesta:

```
% Escribe aquí el comando
```

Listing 2.1: Comando para listar dependencias disponibles.

2.2 Crear un Mensaje

Pregunta 10

¿Qué comando se debe utilizar para saber de qué tipo es el mensaje creado?

Respuesta:

```
% Escribe aquí el comando
```

Listing 2.2: Comando para verificar el tipo de mensaje MiMensaje.

Pregunta 11

Muestra el gráfico de entorno ROS generado usando `rqt_graph`.

[Captura rqt_graph Parte 2]

Figura 2.1: Gráfico `rqt_graph` con los nodos `nodo_envia` y `nodo_recibe`.

Pregunta 12

Intenta modificar la constante nombre del mensaje, modificando el código de `nodo_envia`. ¿Qué sucede?

Respuesta:

Pregunta 13

¿De qué tipo es el topic que aparece?

Respuesta:

2.3 Ejercicios

2.3.1 Ejercicio 1: Paquete `p2pkg` y mensaje `P2pkg_mensaje`

Crea el paquete `p2pkg` y define en `interfaz` el mensaje `P2pkg_mensaje.msg`.

Descripción:

```
cd /workspace/ros2_ws/src
ros2 pkg create --build-type ament_python --license Apache-2.0
   p2pkg --dependencies rclpy std_msgs
```

Listing 2.3: Creación del paquete p2pkg.

Campos del mensaje `P2pkg_mensaje.msg`:

```
int32 numero
geometry_msgs/Pose posicion
string fecha
```

Listing 2.4: Definición del mensaje P2pkg_mensaje.msg.

Resultado del comando de verificación:

2.3.2 Ejercicio 2: Nodos publisher y subscriber en `p2pkg`

Nodo publisher `nodopub_ejercicio2` *y subscriber* `nodosub_ejercicio2` *para el topic* `/topic_ejercicio2`*.*

Descripción de la implementación:

3. Parte 3: Pasando a Nivel Intermedio

3.1 Crear un archivo launch

Pregunta 14

¿Con qué nombre se han lanzado los nodos?

Respuesta:

3.2 Acciones

Pregunta 15

¿Cómo podemos saber que se ha lanzado la acción correctamente? ¿Qué método permite saber que la acción ha finalizado exitosamente?

Respuesta:

Pregunta 16

Sin crear un nodo, ¿cómo lanzarías la acción Fibonacci? ¿Cómo observarías el resultado?

Respuesta:

```
% Escribe aquí el comando ros2 action send_goal
```

Listing 3.1: Lanzar la acción Fibonacci desde terminal.

Pregunta 17

Si paramos el nodo a mitad de la ejecución de la acción, ¿se cancela la acción? ¿Se puede cancelar la acción desde la terminal?

Respuesta:

3.3 Ejercicios

3.3.1 Ejercicio 1: `launch_ejercicio3.py`

Fichero launch que lanza los nodos del ejercicio anterior con argumento de entrada (valor por defecto 7).

Descripción:

3.3.2 Ejercicio 2: `launch_ejercicio4.py` con namespace y remapping

Fichero launch que agrupa los nodos en `miGrupo` con remapping del topic.

Descripción:

3.3.3 Ejercicio 3: Acción `ejFibonacci.action`

Nueva acción con el mismo goal y result que `Fibonacci.action`, pero con feedback de tipo `float64`.

Definición del fichero `ejFibonacci.action`:

```
int32 orden
---
int32[] secuencia_final
---
float64 feedback_valor
```

Listing 3.2: Definición de ejFibonacci.action.

3.3.4 Ejercicio 4: Servidor `ejercicios_fibServer.py`

Servidor de `ejFibonacci` cuyo feedback es la raíz cuadrada de la media de la secuencia calculada.

Descripción:

3.3.5 Ejercicio 5: Cliente `ejercicios_fibClient.py`

Cliente que acepta el orden por parámetro y publica 'en proceso' al topic `/estado_accion`.

Descripción:

3.3.6 Ejercicio 6: Servicio `service_temp` (conversión de temperatura)

Servicio que convierte entre Celsius y Fahrenheit.

Definición del fichero `.srv`:

```
float64 input_temp
string conversion_type
---
float64 converted_temp
```

Listing 3.3: Interfaz del servicio service_temp.

Descripción de la implementación:

3.3.7 Ejercicio 7: Acción `battery_act` (simulación de batería)

Sistema de acciones que simula la descarga de batería de un robot.

Definición del fichero `.action`:

```
int32 target_percentage
---
string warning
---
int32 current_percentage
```

Listing 3.4: Interfaz de la acción battery_act.

Descripción: